

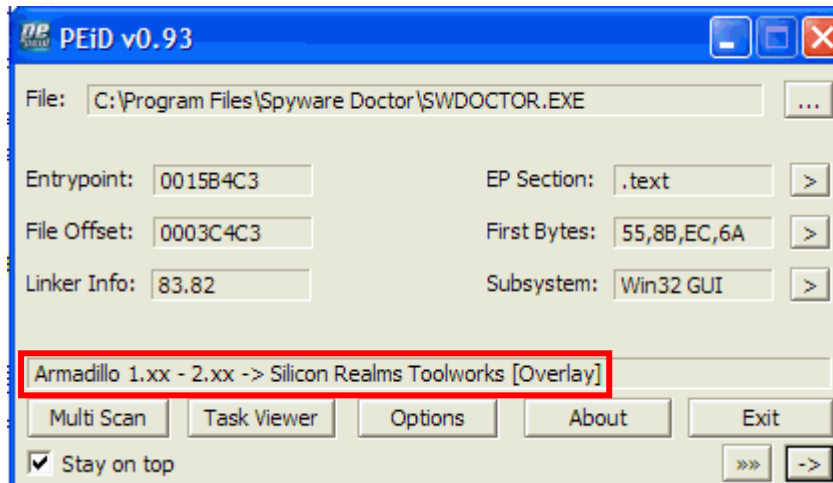
악성코드 분석 보고서

(sand-reversingwithlana-tutorials)

2025.10.13

24와 마찬가지로 실행이되지 않아 lena영상 및 다른 사용자의 글을 참고하여 작성하였다.

1) SpywareDoctor.exe – Loader 사용



아르마딜로 패커를 사용한 걸 볼 수 있다.



Register Spyware Doctor

Enhanced program features require a licensed version of Spyware Doctor.

To upgrade to the licensed version, please click Purchase Online or click Continue to access the free version. Registered users, please enter your registration and license details below to activate the licensed version.

License Name

License Code

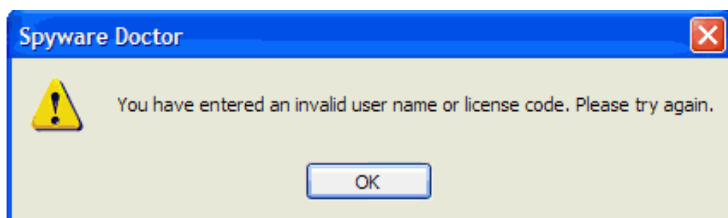
[Need help registering?](#)

Register Now

Purchase Online

Continue

레지스터 값을 입력하는 창이 생기는데 값을 입력해주고 'Register Now'를 눌러준다.



이렇게 입력한 값이 틀리다는 걸 알 수 있다.

K Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	Frame
0012BBB8	77D19418	Includes ntdll.KiFastSystemCallRet	USER32.77D19416	0012BBEC
0012BBBC	77D2E2A2	USER32.WaitMessage	USER32.77D2E290	0012BBEC
0012BBF0	77D261C6	USER32.77D2E113	USER32.77D261C1	0012BBEC
0012BC18	77D3A92E	USER32.77D26110	USER32.77D3A929	0012BC14
0012BED8	77D3A294	USER32.SoftModalMessageBox	USER32.77D3A28F	0012BED4
0012C028	77D65FB6	USER32.77D3A11F	USER32.77D65FB6	0012C024
0012C080	77D66060	USER32.MessageBoxTimeoutW	USER32.77D6605B	0012C07C
0012C0B4	77D50577	? USER32.MessageBoxTimeoutA	USER32.77D50572	0012C0B0
0012C0D4	77D5052F	? USER32.MessageBoxExA	USER32.77D5052A	0012C0D0
0012C0D8	009F0298	hOwner = 009F0298 ('Spyware Doctor',class='TApplication')		
0012C0DC	01A09D68	Text = "You have entered an invalid user name or license code"		
0012C0E0	04E38384	Title = "Spyware Doctor"		
0012C0E4	00040030	Style = MB_OK MB_ICONEXCLAMATION MB_APPLMODAL:40000		
0012C0E8	00000000	LanguageID = 0 (LANG_NEUTRAL)		

올리디버그로 돌아간 후 콜스택을 보면 정보가 없는 걸 확인할 수 있다.

하지만 키 값을 입력한 후 'Register Now'를 누르면 bad boy창이 늦게 뜨는 걸 알 수 있다. 그래서 오래 걸리는 시간을 이용하여 일시정지를 해주고 콜 스택을 열어보도록 한다.

* 올리디버그 단축키

- Alt + F9 : 시스템/패커 루틴을 건너뛰고 사용자 코드 진입점으로 실행 (커서가 있는 부분에서 멈춤)

- Ctrl + F9 : 현재 함수가 RET 명령으로 끝날 때까지 실행

- Shift+ F9 : 시스템 DLL 내부에서 멈췄을 때, 바로 사용자 코드로 복귀해서 거기서 멈춤
- F9 : 프로그램 전체 계속 실행 (다음 BP까지)

K Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	Frame
001286A8	7C949FAC	ntdll.RtlFillMemoryUlong	ntdll.7C949FA7	001288D4
001288D8	7C96D6AA	? ntdll.7C91B2AC	ntdll.7C96D6A5	001288D4
0012895C	7C949D18	? ntdll.7C96D5F8	ntdll.7C949D13	00128958
00128B8C	7C91B298	? ntdll.7C91B2AC	ntdll.7C91B293	00128B88
00128DC0	77BFC3C9	? ntdll.RtlAllocateHeap	msvcrt.77BFC3C3	00128DBC
00128DC4	003D0000	hHeap = 003D0000		
00128DC8	40000060	Flags = HEAP_TAIL_CHECKING_ENABLED;HEAP_FREE_CHECKING_ENABLED		
00128DCC	0000000A	HeapSize = A (10.)		
00128E00	77BFC3E7	? msvcrt.77BFC2E9	msvcrt.77BFC3E2	00128DFC
00128E0C	77BF9CD4	msvcrt.77BFC3D4	msvcrt.77BF9CCF	00128E08
00128E1C	00E61FEB	00E91370	00E61FE6	00128E18
00128E2C	00E621AF	00E61FD4	00E621AA	00128E54
00128E3C	00E6229C	00E6218E	00E62297	00128E54
00128E58	00E62F3E	00E62874	00E62F39	00128E54
00128E78	00E62E37	00E62F12	00E62E32	00128E74
00128ED8	00E6342F	00E62B8F	00E6342A	00128ED4
00128F00	00E633CC	00E633F6	00E633C7	00128EFC
00128F34	00E63586	00E632F5	00E63581	00128F30
00128F80	00E771C4	00E63465	00E771BF	00128F7C
00129324	00E79D45	00E76D27	00E79D40	00129320
001296A4	00E7969F	00E7998C	00E7969A	001296A0
00129C08	00E791CB	00E791EF	00E791C6	00129C04
00129F1C	00E7B497	00E7914A	00E7B492	00129F18
0012BF48	00E7B3E1	00E7B44F	00E7B3DC	0012BF44
0012BF5C	00E8B8B8	00E7B3BC	00E8B8B6	0012C174
0012C178	00E8B8B8	00E8B75B	00E8B75B	0012C174
0012C18C	00465F1D	swdoctor.00464AF0	swdoctor.00465F18	0012C1D0
0012C190	04E000F0	Arg1 = 04E000F0 ASCII "lenal51"		
0012C194	04E38384	Arg2 = 04E38384 ASCII "55555555555555555555555555555555"		

이렇게 입력한 값이 swdocotr.00464AF0에 저장된 걸 볼 수 있고 0x00465F18에서 해당 함수가 불러진 걸 볼 수 있다. 0x00465F18 위치로 들어가본다.

CPU - main thread, module swdoctor				
00465F18	E8 D3EBFFFF	CALL swdoctor.00464AF0		
00465F1D	84C8	TEST AL,AL		
00465F1F	74 3B	JE SHORT swdoctor.00465F5C		
00465F21	8D55 EC	LEA EDX, DWORD PTR SS:[EBP-14]		
00465F24	8B45 FC	MOV EAX, DWORD PTR SS:[EBP-4]		
00465F27	8B80 10030000	MOV EAX, DWORD PTR DS:[EAX+310]		
00465F2D	E8 4EDAF9FF	CALL swdoctor.00403980		JMP to vc170.Controls::TControl::GetText
00465F32	8B45 EC	MOV EAX, DWORD PTR SS:[EBP-14]		
00465F35	E8 56B3F9FF	CALL swdoctor.00401290		JMP to rtl70.System::LStrToPChar
00465F3A	50	PUSH EAX		
00465F3B	8D55 E8	LEA EDX, DWORD PTR SS:[EBP-18]		
00465F3E	8B45 FC	MOV EAX, DWORD PTR SS:[EBP-4]		
00465F41	8B80 0C030000	MOV EAX, DWORD PTR DS:[EAX+30C]		
00465F47	E8 34DAF9FF	CALL swdoctor.00403980		JMP to vc170.Controls::TControl::GetText
00465F4C	8B45 E8	MOV EAX, DWORD PTR SS:[EBP-18]		
00465F4F	E8 3CB3F9FF	CALL swdoctor.00401290		JMP to rtl70.System::LStrToPChar
00465F54	50	PUSH EAX		
00465F55	E8 9EEBFFFF	CALL swdoctor.00464AF8		
00465F5A	EB 05	JMP SHORT swdoctor.00465F61		
00465F5C	E8 AFBFFFFF	CALL swdoctor.00464B10		

※ 리버싱(역공학)은 약간의 '시행착오(trial and error)'가 섞여 있다. 하지만 리버싱의 진짜 묘미는 '추측'을 가능한 한 줄이고 논리적인 관찰로 추측해야한다. 조금만 살펴보면 (지금 처럼 압축 해제된 소프트웨어에서도 볼 수 있듯이), 이 부분이 등록 루틴(registration routine) 이라는 걸 꽤 높은 확신으로 알 수 있다.

00465EEA	8B85	CALL swdoctor.00403980	JMP to vc170.Controls::TControl::GetText
00465EF0	E8 8BDAF9FF	MOV EAX,DWORD PTR SS:[EBP-C]	ntdll.7C90EE18
00465EF5	8B45 F4	CALL swdoctor.00401290	JMP to rtl70.System::LStrToPChar
00465EF8	E8 93B3F9FF	PUSH EAX	
00465EFA	50		
00465EFE	8055 F0	LEA EDX,DWORD PTR SS:[EBP-10]	
00465F01	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465F04	8B80 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]	
00465F08	E8 71DAF9FF	CALL swdoctor.00403980	JMP to vc170.Controls::TControl::GetText
00465F0F	8B45 F0	MOV EAX,DWORD PTR SS:[EBP-10]	
00465F12	E8 79B3F9FF	CALL swdoctor.00401290	JMP to rtl70.System::LStrToPChar
00465F17	50	PUSH EAX	
00465F18	E8 D3EBFFFF	CALL swdoctor.00464AF0	
00465F1D	84C0	TEST AL,AL	
00465F1F	74 3B	JE SHORT swdoctor.00465F5C	
00465F21	8055 EC	LEA EDX,DWORD PTR SS:[EBP-14]	
00465F24	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465F27	8B80 10030000	MOV EAX,DWORD PTR DS:[EAX+310]	
00465F2D	E8 4EDAF9FF	CALL swdoctor.00403980	JMP to vc170.Controls::TControl::GetText
00465F32	8B45 EC	MOV EAX,DWORD PTR SS:[EBP-14]	
00465F35	E8 56B3F9FF	CALL swdoctor.00401290	JMP to rtl70.System::LStrToPChar
00465F3A	50	PUSH EAX	
00465F3B	8055 E8	LEA EDX,DWORD PTR SS:[EBP-18]	
00465F3E	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465F41	8B80 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]	
00465F47	E8 34DAF9FF	CALL swdoctor.00403980	JMP to vc170.Controls::TControl::GetText

스크롤을 올리면 빨간색의 첫 번째 GetText에서는 시리얼을 읽는 부분, 두 번째 GetText는 이름을 읽는 부분이라는 걸 추정해볼 수 있고 그 후 값을 받아서 0x00464AF0으로 넘어가는 걸 볼 수 있고 그 후 JE 분기문이 실행되는 걸 볼 수 있다.

그리고 파란색의 첫 번째와 두 번째도 시리얼과 이름을 읽는 부분이라는 걸 추정할 수 있다. 하지만 JE가 실행된다면 이 부분은 실행되지 않을 것이다.

- GetText의 역할

GetText는 거의 항상 텍스트 속성을 읽어오는 메서드.

즉, 프로그램의 UI 요소(텍스트 박스, 레이블, 버튼 등)에 표시된 문자열을 가져오는 함수.

00465EC0	55	PUSH EBP	
00465EC1	8BEC	MOV EBP,ESP	
00465EC2	8B 05000000	MOV ECX,EAX	
00465EC8	6A 00	PUSH 0	
00465ECA	6A 00	PUSH 0	
00465ECC	49	DEC ECX	
00465ECD	75 F9	JNZ SHORT swdoctor.00465EC8	
00465ECF	51	PUSH ECX	
00465ED0	8955 F8	MOV DWORD PTR SS:[EBP-8],EDX	
00465ED3	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	

스크롤을 올리면 해당 부분을 볼 수 있는데 단지 스택을 준비하는(pushing 0) 동작을 하는 부분이다.

00465F6E	FF50 0C	CALL DWORD PTR DS:[EAX+C]
00465F71	84C0	TEST AL,AL
00465F73	75 52	JNZ SHORT swdoctor.00465FC7
00465F75	58 4604600	PUSH swdoctor.00466040
00465F7A	68 4C604600	PUSH swdoctor.00466040
00465F7F	E8 6CEBFFFF	CALL swdoctor.00464AF0
00465F84	8D4D E4	LEA ECX,DWORD PTR SS:[EBP-1C]
00465F87	A1 F0995000	MOV EAX,DWORD PTR DS:[5099F0]
00465F8C	8B00	MOV EAX,DWORD PTR DS:[EAX]
00465F8E	BA 58604600	MOV EDX,swdoctor.00466058
00465F93	E8 3481FBFF	CALL swdoctor.0041E0C0
00465F98	8B45 E4	MOV EAX,DWORD PTR SS:[EBP-1C]
00465F9B	BA 30000400	MOV EDX,40030
00465FA0	E8 971DFAFF	CALL swdoctor.00407D3C
00465FA5	33D2	XOR EDX,EDX
00465FA7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]
00465FAA	8B80 10030000	MOV EAX,DWORD PTR DS:[EAX+310]
00465FB0	E8 D3D9F9FF	CALL swdoctor.00403988
00465FB5	33D2	XOR EDX,EDX
00465FB7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]
00465FBA	8B80 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]
00465FC0	E8 C3D9F9FF	CALL swdoctor.00403988
00465FC5	EB 4F	JMP SHORT swdoctor.00466016
00465FC7	8D45 E0	LEA EAX,DWORD PTR SS:[EBP-20]
00465FCA	50	PUSH EAX
00465FCB	8D4D DC	LEA ECX,DWORD PTR SS:[EBP-24]
00465FCE	01 F0995000	MOVB BYTE PTR DS:[5099F0]

Ah, but this is where it is decided !

ASCII "Msg29"

JMP to vc170.Controls::TControl::SetText

JMP to vc170.Controls::TControl::SetText

콜 스택에서 0x00465F18 기준으로 밑으로 내려가보면 또 하나의 분기문이 있는 걸 볼 수 있다. 해당 분기문이 점프를 하면 SetText가 실행 안되는 걸 볼 수 있다.

해당 부분은 실패 메시지를 보여주는 함수이므로 점프를 해야 성공 메시지를 띄울 수 있다는 것을 추정할 수 있다.

해당 분기문에서 메시지의 방향성을 선택하니까 그 위의 call 함수인 0x00465F6E는 시리얼과/이름을 비교해서 서로 맞으면 1, 다르면 0을 나오게 하는 걸 알 수 있다.

리턴 값이 1일 경우 분기되는 "465FC7" 주소를 등록 성공이라고 추정하였으니 실제로 맞는지 안 맞는지 테스트 해봐야 한다.

- SetText의 역할

UI 컨트롤의 "Text" 속성 값을 바꾸는 함수.

00465EC0	51	PUSH EBP
00465EC1	8BEC	MOV EBP,ESP
00465EC3	B9 05000000	MOV ECX,5
00465EC8	6A 00	PUSH 0
00465ECA	6A 00	PUSH 0
00465ECC	49	DEC ECX
00465ECD	75 F9	JNZ SHORT swdoctor.00465EC8
00465ECF	51	PUSH ECX
00465ED0	8955 F8	MOV DWORD PTR SS:[EBP-8],EDX
00465ED3	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX
00465ED6	33C0	XOR EAX,EAX
00465ED8	55	PUSH EBP
00465ED9	5D	POP EBP

추정한 것들이 맞는지 확인하기 위해서 처음부터 실행해봐야한다. 0x00465EC0 부분에 HW BP를 건 후 실행해준다.

Register Spyware Doctor

Enhanced program features require a licensed version of Spyware Doctor.

To upgrade to the licensed version, please click Purchase Online or click Continue to access the free version. Registered users, please enter your registration and license details below to activate the licensed version.

License Name

License Code

[Need help registering?](#)

Register Now

Purchase Online

Continue

00465EBF	90	NOP	
00465EC0	55	PUSH EBP	
00465EC1	8BEC	MOV EBP,ESP	
00465EC3	B9 05000000	MOV ECX,5	
00465EC8	6A 00	PUSH 0	
00465ECA	6A 00	PUSH 0	
00465ECC	49	DEC ECX	
00465ECD	75 F9	JNZ SHORT swdoctor.00465EC8	vol170.StdCtrls::TButton::CNCommand
00465ECF	51	PUSH ECX	vol170.StdCtrls::TButton::CNCommand
00465ED0	8955 F8	MOV DWORD PTR SS:[EBP-8],EDX	
00465ED3	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	
00465ED6	33C0	XOR EAX,EAX	

값들을 입력 후 'Register Now'를 눌러주면 BP 부분에서 멈추는 걸 볼 수 있다.

00465EE7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465EEA	8B80 10030000	MOV EAX,DWORD PTR DS:[EAX+310]	
00465EF0	E8 8BDAF9FF	CALL swdoctor.00403980	JMP to vol170.Controls::TControl::GetText
00465EF5	8B45 F4	MOV EAX,DWORD PTR SS:[EBP-C]	
00465EF8	E8 93B3F9FF	CALL swdoctor.00401290	JMP to rtl70.System::LStrToPChar
00465FF0	50	PUSH FAX	
Stack SS:[0012C1C4]=04E38384, (ASCII "66666666666666666666666666666666")			
EAX=0000001A			

아까 빨간색의 첫 번째 부분을 보면 시리얼 값을 읽어드리는 걸 볼 수 있다.

00465F04	8B80 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]	
00465F0A	E8 71DAF9FF	CALL swdoctor.00403980	JMP to vol170.Controls::TControl::GetText
00465F0F	8B45 F0	MOV EAX,DWORD PTR SS:[EBP-10]	
00465F12	E8 79B3F9FF	CALL swdoctor.00401290	JMP to rtl70.System::LStrToPChar
Stack SS:[0012C1C0]=04E539EC, (ASCII "lena151")			
EAX=00000007			

아까 빨간색의 두 번째 부분을 보면 이름 값을 읽어드리는 걸 볼 수 있다.

00465F10	84C0	TEST AL,AL	
00465F1F	74 3B	JE SHORT swdoctor.00465F5C	
00465F21	8D55 EC	LEA EDX,DWORD PTR SS:[EBP-14]	
00465F24	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465F27	8B80 10030000	MOV EAX,DWORD PTR DS:[EAX+310]	
00465F2D	E8 4EDAF9FF	CALL swdoctor.00403980	JMP to vol170.Controls::TControl::GetText
00465F32	8B45 EC	MOV EAX,DWORD PTR SS:[EBP-14]	
00465F35	FA 56R3F9FF	CALL swdoctor.00401290	JMP to rtl70.System::LStrToPChar
Jump is taken			
00465F5C=swdoctor.00465F5C			

그 후 파란색 부분인데 이 부분은 점프하는 걸 볼 수 있다. 해당 부분도 GetText를 사용하는 걸 볼 수 있는데 이 부분은 값을 입력 안하면 발생하는 코드로 알 수 있다.

00465F6C	8B00	MOV EAX,DWORD PTR DS:[EAX]	
00465F6E	FF50 0C	CALL DWORD PTR DS:[EAX+C]	
00465F71	84C0	TEST AL,AL	
00465F73	75 52	JNZ SHORT swdoctor.00465FC7	
00465F75	68 4C604600	PUSH swdoctor.0046604C	
00465F7A	68 4C604600	PUSH swdoctor.0046604C	
00465F7F	E8 6CEBFFFF	CALL swdoctor.00464AF0	
00465F84	8D4D E4	LEA ECX,DWORD PTR SS:[EBP-1C]	
00465F87	A1 F0995000	MOV EAX,DWORD PTR DS:[5099F0]	
00465F8C	8B00	MOV EAX,DWORD PTR DS:[EAX]	
00465F8E	BA 58604600	MOV EDI,swdoctor.00466058	ASCII "Msg29"
00465F93	E8 3481FBFF	CALL swdoctor.0041E0CC	
00465F98	8B45 E4	MOV EAX,DWORD PTR SS:[EBP-1C]	
00465F9B	BA 30000400	MOV EDI,40030	
00465FA0	E8 971DFAFF	CALL swdoctor.00407D3C	rtl70.400765D4
00465FA5	33D2	XOR EDI,EDI	
00465FA7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465FAA	8B00 10030000	MOV EAX,DWORD PTR DS:[EAX+310]	
00465FB0	E8 D3D9F9FF	CALL swdoctor.00403988	JMP to vc170.Controls::TControl::SetText
00465FB5	33D2	XOR EDI,EDI	rtl70.400765D4
00465FB7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465FBA	8B00 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]	
00465FC0	E8 C3D9F9FF	CALL swdoctor.00403988	JMP to vc170.Controls::TControl::SetText
00465FC5	EB 4F	JMP SHORT swdoctor.00466016	

Is this indeed
a bad sign ?

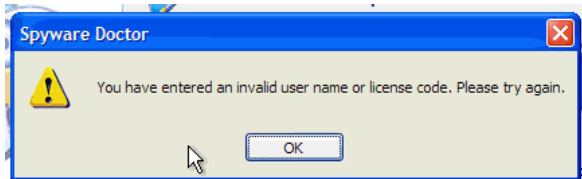
실행하다보면 SetText를 실행하는지 안하는지 판단하는 분기문에 도달하는 걸 볼 수 있다. 전에는 점프를 하면 성공 메시지, 점프를 하지 않으면 실패 메시지를 발생시킨다고 추정을 했는데 이번에 실행해보니까 점프가 실행안되는 걸 판단할 수 있다.

즉, 해당 부분은 추정하는게 맞았고 call함수가 판단하는 부분이라는 걸 알았다.

00465F8C	8B00	MOV EAX,DWORD PTR DS:[EAX]	
00465F8E	BA 58604600	MOV EDI,swdoctor.00466058	ASCII "Msg29"
00465F93	E8 3481FBFF	CALL swdoctor.0041E0CC	
00465F98	8B45 E4	MOV EAX,DWORD PTR SS:[EBP-1C]	
00465F9B	BA 30000400	MOV EDI,40030	
00465FA0	E8 971DFAFF	CALL swdoctor.00407D3C	swdoctor.00466058
00465FA5	33D2	XOR EDI,EDI	
00465FA7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465FAA	8B00 10030000	MOV EAX,DWORD PTR DS:[EAX+310]	
00465FB0	E8 D3D9F9FF	CALL swdoctor.00403988	JMP to vc170.Controls::TControl::SetText
00465FB5	33D2	XOR EDI,EDI	swdoctor.00466058
00465FB7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465FBA	8B00 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]	
00465FC0	E8 C3D9F9FF	CALL swdoctor.00403988	JMP to vc170.Controls::TControl::SetText
00465FC5	EB 4F	JMP SHORT swdoctor.00466016	
00465FC7	8D45 E0	LEA EAX,DWORD PTR SS:[EBP-20]	
00465FCA	50	PUSH EAX	
00465FCB	8D4D DC	LEA ECX,DWORD PTR SS:[EBP-24]	
00465FCE	A1 F0995000	MOV EAX,DWORD PTR DS:[5099F0]	
00465FD3	8B00	MOV EAX,DWORD PTR DS:[EAX]	
00465FD5	BA 68604600	MOV EDI,swdoctor.00466068	ASCII "Msg28"
00465FDA	E8 ED80FBFF	CALL swdoctor.0041E0CC	
00465FDF	8B45 DC	MOV EAX,DWORD PTR SS:[EBP-24]	
00465FE2	50	PUSH EAX	
00465FE3	A1 8C985000	MOV EAX,DWORD PTR DS:[50988C]	
00465FE8	8B00	MOV EAX,DWORD PTR DS:[EAX]	
00465FEA	8945 D4	MOV DWORD PTR SS:[EBP-2C],EAX	
00465FED	C645 D8 0B	MOV BYTE PTR SS:[EBP-28],0B	
00465FF1	8D55 D4	LEA EDI,DWORD PTR SS:[EBP-2C]	
00465FF4	33C9	XOR ECX,ECX	
00465FF6	5A	POP EAX	0012C500

Preparing the
Badboy !

Stack SS:[0012C1B4]=01A09D68, (ASCII "You have entered an invalid user name or license code. Please try again.")
EAX=0012C188



실행을 해보면 발생시키는 걸 볼 수 있다. 다시 올리디버그로 돌아가본다.

00465FA0	E8 971DFAFF	CALL swdoctor.00407D3C	
00465FA5	33D2	XOR EDI,EDI	
00465FA7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465FAA	8B00 10030000	MOV EAX,DWORD PTR DS:[EAX+310]	
00465FB0	E8 D3D9F9FF	CALL swdoctor.00403988	JMP to vc170.Controls::TControl::SetText
00465FB5	33D2	XOR EDI,EDI	
00465FB7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00465FBA	8B00 0C030000	MOV EAX,DWORD PTR DS:[EAX+30C]	
00465FC0	E8 C3D9F9FF	CALL swdoctor.00403988	JMP to vc170.Controls::TControl::SetText
00465FC5	EB 4F	JMP SHORT swdoctor.00466016	

0x00465FA0에서 실패 메시지가 발생하는 걸 볼 수 있다. 아까 경고 창에서 'OK'버튼을

누르면 "SetText" 함수를 호출하여 입력 창을 비워서 사용자의 입력을 기다리는 걸 볼 수 있다.

00465F6B	50	PUSH EAX
00465F6C	8B00	MOV EAX,DWORD PTR DS:[EAX]
00465F6E	FF50 0C	CALL DWORD PTR DS:[EAX+C]
00465F71	84C0	TEST AL,AL
00465F73	75 52	JNZ SHORT swdoctor.00465FC7
00465F75	68 4C604600	PUSH swdoctor.0046604C
00465F7A	68 4C604600	PUSH swdoctor.0046604C
00465F7F	E8 6CEBFFFF	CALL swdoctor.00464AF0
00465F84	8D4D E4	LEA ECX,DWORD PTR SS:[EBP-1C]
00465F87	A1 F0995000	MOV EAX,DWORD PTR DS:[5099F0]
00465F8C	8B00	MOV EAX,DWORD PTR DS:[EAX]
00465F8E	BA 58604600	MOV EDI,swdoctor.00466058
00465F93	E8 3481FBFF	CALL swdoctor.0041E0CC
00465F98	8B45 E4	MOV EAX,DWORD PTR SS:[EBP-1C]
00465F9B	BA 30000400	MOV EDI,40030

JNZ -> JMP로 바꾸면 되겠지만 이번에는 조건문(JNZ)에 사용되는 AL 레지스터 값을 세팅하는 0x00465F6E를 패치하는 "intermediate" 패치를 보여줄 것이다.

왜 이렇게 복잡한 패치를 진행하는지 궁금하다면, 만약 "JNZ" 조건문을 패치해 버리면 그냥 메인 루틴 부분만 패치되는 것이다. 따라서 다른 루틴에서 해당 함수를 호출해 버리면 무용 지물이 된다. 그러나 "[EAX+C]" 함수 내부에서 AL 레지스터 값을 패치해 버리면 다른 루틴에서 해당 함수를 호출해도 자동적으로 패치가 된다.



다시 값들을 입력 후 'Regiser Now'를 눌러준 후 0x00465F6E위치까지 와준다.

00465F6B	50	PUSH EAX	
00465F6C	8B00	MOV EAX,DWORD PTR DS:[EAX]	swdoctor.00502995
00465F6E	FF50 0C	CALL DWORD PTR DS:[EAX+C]	swdoctor.0050296D
00465F71	84C0	TEST AL,AL	
00465F73	75 52	JNZ SHORT swdoctor.00465FC7	
00465F75	68 4C604600	PUSH swdoctor.0046604C	
00465F7A	68 4C604600	PUSH swdoctor.0046604C	
00465F7F	E8 6CEBFFFF	CALL swdoctor.00464AF0	
00465F84	8D4D E4	LEA ECX,DWORD PTR SS:[EBP-1C]	
00465F87	A1 F0995000	MOV EAX,DWORD PTR DS:[5099F0]	
00465F8C	8B00	MOV EAX,DWORD PTR DS:[EAX]	swdoctor.00502995
00465F8E	BA 58604600	MOV EDI,swdoctor.00466058	ASCII "Msg29"
00465F93	E8 3481FBFF	CALL swdoctor.0041E0CC	
00465F98	8B45 E4	MOV EAX,DWORD PTR SS:[EBP-1C]	
00465F9B	BA 30000400	MOV EDI,40030	

해당 부분을 'F7'을 눌러 들어가준다.

0050296D	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
00502972	E9 39060000	JMP swdoctor.00502FB0
00502977	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
0050297C	E9 4B0B0000	JMP swdoctor.005029CC
00502981	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
00502986	E9 D90B0000	JMP swdoctor.00502964
0050298B	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
00502990	E9 F3060000	JMP swdoctor.00502988
00502995	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
0050299A	E9 89EAEFFF	JMP swdoctor.00401228
0050299F	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
005029A4	E9 87EAEFFF	JMP swdoctor.00401230
005029A9	834424 04 F4	ADD DWORD PTR SS:[ESP+4],-0C
005029AE	E9 85EAEFFF	JMP swdoctor.00401238

0x0050296D에 도달하고 'F8'을 이용해 하나 내려가 주면 JMP분기문이 실행되는 걸 볼 수 있을 것이다.

00502FB0	55	PUSH EBP	
00502FB1	8BEC	MOV EBP,ESP	
00502FB3	6A 00	PUSH 0	
00502FB5	6A 00	PUSH 0	
00502FB7	6A 00	PUSH 0	
00502FB9	33C0	XOR EAX,EAX	swdoctor.005029B4
00502FBB	55	PUSH EBP	
00502FBC	68 43305000	PUSH DWORD PTR FS:[EAX]	
00502FBD	64:FF30	PUSH EDI	
00502FBE	64:8920	MOV DWORD PTR FS:[EAX],ESP	
00502FC0	8D55 F8	LEA EDX,DWORD PTR SS:[EBP-8]	
00502FC1	B8 54305000	MOV EAX,swdoctor.00503054	ASCII "USERNAME"
00502FC2	E8 BCFCFFFF	CALL swdoctor.00502C90	
00502FC4	837D F8 00	CMP DWORD PTR SS:[EBP-8],0	
00502FC5	74 22	JE SHORT swdoctor.00502FFC	
00502FC6	BA 68305000	MOV EDX,swdoctor.00503068	ASCII "DEFAULT"
00502FC7	8B45 F8	MOV EAX,DWORD PTR SS:[EBP-8]	
00502FC8	E8 59EFEFFF	CALL swdoctor.00401F40	
00502FC9	85C0	TEST EAX,EAX	
00502FCA	74 11	JE SHORT swdoctor.00502FFC	
00502FCB	BA 78305000	MOV EDX,swdoctor.00503078	
00502FCC	8B45 F8	MOV EAX,DWORD PTR SS:[EBP-8]	
00502FCD	E8 48EFEFFF	CALL swdoctor.00401F40	
00502FCE	85C0	TEST EAX,EAX	
00502FCF	75 06	JNZ SHORT swdoctor.00503002	
00502FD0	C645 FF 00	MOV BYTE PTR SS:[EBP-14],0	
00502FD1	EB 04	JMP SHORT swdoctor.00503006	
00502FD2	C645 FF 01	MOV BYTE PTR SS:[EBP-13],1	
00502FD3	807D FF 00	CMP BYTE PTR SS:[EBP-13],0	
00502FD4	74 13	JE SHORT swdoctor.0050301F	
00502FD5	8D55 F4	LEA EDX,DWORD PTR SS:[EBP-C]	
00502FD6	B8 80305000	MOV EAX,swdoctor.00503080	
00502FD7	E8 77FCFFFF	CALL swdoctor.00502C90	
00502FD8	837D F4 00	CMP DWORD PTR SS:[EBP-C],0	
00502FD9	75 04	JNZ SHORT swdoctor.00503023	
00502FDA	33C0	XOR EAX,EAX	
00502FDB	EB 02	JMP SHORT swdoctor.00503025	
00502FDC	B0 01	MOV AL,1	
00502FDD	8845 FF	MOV BYTE PTR SS:[EBP-1],1	
00502FDE	33C0	XOR EAX,EAX	
00502FDF	5A	POP EDX	
00502FE0	59	POP ECX	
00502FE1	59	POP ECX	
00502FE2	84 0910	MOV DWORD PTR FS:[EAX],EBP	
00502FE3	68 4A305000	PUSH DWORD PTR FS:[EAX]	
00502FE4	8D45 F4	LEA EAX,DWORD PTR SS:[EBP-C]	
00502FE5	BA 02000000	MOV EDI,2	
00502FE6	E8 CEE1EFFF	CALL swdoctor.00401210	
00502FE7	C3	RETN	

And we drop in
this small routine.

0x0050296D에 HW BP를 설치하고 프로그램을 재실행해보니 해당 부분이 총 23번 호출되었다. 23번 호출되었다는 의미는 등록 상태를 검사하는 코드가 23번 사용되었다는 것이다.

00502FBB	55	PUSH EBP	
00502FBC	68 43305000	PUSH DWORD PTR FS:[EAX]	
00502FBD	64:FF30	PUSH EDI	
00502FBE	64:8920	MOV DWORD PTR FS:[EAX],ESP	
00502FC0	8D55 F8	LEA EDX,DWORD PTR SS:[EBP-8]	
00502FC1	B8 54305000	MOV EAX,swdoctor.00503054	
00502FC2	E8 BCFCFFFF	CALL swdoctor.00502C90	
00502FC4	837D F8 00	CMP DWORD PTR SS:[EBP-8],0	
00502FC5	74 22	JE SHORT swdoctor.00502FFC	
00502FC6	BA 68305000	MOV EDX,swdoctor.00503068	
00502FC7	8B45 F8	MOV EAX,DWORD PTR SS:[EBP-8]	
00502FC8	E8 59EFEFFF	CALL swdoctor.00401F40	
00502FC9	85C0	TEST EAX,EAX	
00502FCA	74 11	JE SHORT swdoctor.00502FFC	
00502FCB	BA 78305000	MOV EDX,swdoctor.00503078	
00502FCC	8B45 F8	MOV EAX,DWORD PTR SS:[EBP-8]	
00502FCD	E8 48EFEFFF	CALL swdoctor.00401F40	
00502FCE	85C0	TEST EAX,EAX	
00502FCF	75 06	JNZ SHORT swdoctor.00503002	
00502FD0	C645 FF 00	MOV BYTE PTR SS:[EBP-14],0	
00502FD1	EB 04	JMP SHORT swdoctor.00503006	
00502FD2	C645 FF 01	MOV BYTE PTR SS:[EBP-13],1	
00502FD3	807D FF 00	CMP BYTE PTR SS:[EBP-13],0	
00502FD4	74 13	JE SHORT swdoctor.0050301F	
00502FD5	8D55 F4	LEA EDX,DWORD PTR SS:[EBP-C]	
00502FD6	B8 80305000	MOV EAX,swdoctor.00503080	
00502FD7	E8 77FCFFFF	CALL swdoctor.00502C90	
00502FD8	837D F4 00	CMP DWORD PTR SS:[EBP-C],0	
00502FD9	75 04	JNZ SHORT swdoctor.00503023	
00502FDA	33C0	XOR EAX,EAX	
00502FDB	EB 02	JMP SHORT swdoctor.00503025	
00502FDC	B0 01	MOV AL,1	
00502FDD	8845 FF	MOV BYTE PTR SS:[EBP-1],1	
00502FDE	33C0	XOR EAX,EAX	
00502FDF	5A	POP EDX	
00502FE0	59	POP ECX	
00502FE1	59	POP ECX	
00502FE2	84 0910	MOV DWORD PTR FS:[EAX],EBP	
00502FE3	68 4A305000	PUSH DWORD PTR FS:[EAX]	
00502FE4	8D45 F4	LEA EAX,DWORD PTR SS:[EBP-C]	
00502FE5	BA 02000000	MOV EDI,2	
00502FE6	E8 CEE1EFFF	CALL swdoctor.00401210	
00502FE7	C3	RETN	

'F8'을 이용해서 내려가다 보면 RETN이 있는 걸 볼 수 있다.

해당 빨간 박스를 보면 PUSH를 사용하고 RETN이 나오는 걸 보면 JMP를 의미하는 걸 알 수 있다. (PUSH + RETN == JMP)

※ 원래 PUSH -> CALL -> RETN != JMP 이다.

왜냐하면 CALL함수가 정상적으로 리턴하지 않으면 RETN에 도달하지 않기 때문이다.

1. PUSH addr + RET가 JMP addr인 이유

- PUSH <addr>: 스택에 <addr> 값을 넣음 (스택 포인터 SP가 감소하고 그 위치에 <addr> 저장).

- RET: 스택에서 값 하나를 팝해서(POP) 그 값을 리턴 주소로 사용해 점프함.

따라서 바로 이어서 PUSH target 다음에 RET이 실행되면, RET는 스택에서 target을 꺼내서 그 주소로 점프한다. 결과적으로 push target; ret는 jmp target과 동작이 동일하게 보인다.

2. 중간에 CALL이 있는 경우

- CALL 명령은 현재 위치(리턴 주소)를 스택에 푸시하고 호출한 함수로 점프한다.

그래서 만약 실행 흐름상 PUSH target 이후 사이에서 다른 PUSH/CALL/POP 등이 실행된다면 스택의 최상단(RET가 pop할 값)이 바뀐다.

이러면 RET가 target이 아니라 그 사이에 쌓인 다른 값(예: CALL의 리턴 주소)을 팝해서 그쪽으로 점프하게 되므로 의도한 동작이 깨진다.

-> PUSH target — (CALL 어떤 함수) — RET를 **트램폴린(trampoline)** 형태라고 부린다.

- 트램폴린(trampoline)

트램폴린(trampoline) 은 어셈블리나 시스템 프로그래밍, 함수 호출 구조에서

“제어 흐름을 다른 코드로 ‘튀겨’ 보내는 중간 다리 역할”을 하는 짧은 점프용 코드 조각이다.

00502FB8	55	PUSH EBP	
00502FBC	68 43305000	PUSH sldoctor.00503043	
00502FC1	64:FF30	PUSH DWORD PTR FS:[EAX]	
00502FC4	64:8920	MOV DWORD PTR FS:[EAX],ESP	
00502FC7	8D55 F8	LEA EDX,DWORD PTR SS:[EBP-8]	
00502FCA	B8 54305000	MOV EAX,sldoctor.00503054	ASCII "USERNAME"
00502FCF	E8 BCFCFFFF	CALL sldoctor.00502C90	
00502FD4	837B F8 00	CMPL DWORD PTR SS:[EBP-8],0	
00502FD8	74 22	JE SHORT sldoctor.00502FFC	
00502FDA	BA 68305000	MOV EDX,sldoctor.00503068	ASCII "DEFAULT"
00502FDF	8B45 F8	MOV EAX,DWORD PTR SS:[EBP-8]	
00502FE2	E8 59EFEFFF	CALL sldoctor.00401F40	JMP to rtl70.Sysutils::CompareText
00502FE7	85C0	TEST EAX,EAX	
00502FE9	74 11	JE SHORT sldoctor.00502FFC	
00502FEB	BA 78305000	MOV EDX,sldoctor.00503078	ASCII "MISSING"
00502FF0	8B45 F8	MOV EAX,DWORD PTR SS:[EBP-8]	
00502FF3	E8 4C5EFFFF	CALL sldoctor.00401F40	JMP to rtl70.Sysutils::CompareText
00502FF8	85C0	TEST EAX,EAX	

JE분기문에서 점프 안하는 걸 볼 수 있다. 근데 해당 부분은 CompareText가 있는 부분이다. 그럼 이 부분에서 결정되는 걸 볼 수 있다.

00502FC7	8055 F8	LEA EDX, DWORD PTR SS:[EBP-8]	ASCII "USERNAME"	Registers (FPU) EAX 00000000 ECX 00000000 EDX 00000007 EBX 04DF512C ESP 0012C174 EBP 0012C18C ESI 01711684 EDI 0012C34C EIP 00502FE7 CS 0010 SS 0023 DS 0023 FS 0030 GS 0000
00502FC8	B8 54305000	MOV EAX, swdoctor.00503054		
00502FC9	E8 BCFCFFFF	CALL swdoctor.00502C90		
00502FD4	837D F8 00	CMP DWORD PTR SS:[EBP-8], 0		
00502FD5	74 22	JE SHORT swdoctor.00502FFC		
00502FD6	B8 65305000	MOV EDI, swdoctor.00503065		
00502FD7	8B45 F8	MOV EAX, DWORD PTR SS:[EBP-8]		
00502FE2	E8 59FEFFFF	CALL swdoctor.00401F40		
00502FE3	85C0	TEST EAX, EAX		
00502FE9	74 11	JE SHORT swdoctor.00502FFC		
00502FEB	B8 78305000	MOV EDI, swdoctor.00503078		
00502FF0	8B45 F8	MOV EAX, DWORD PTR SS:[EBP-8]		
00502FF3	E8 48FEFFFF	CALL swdoctor.00401F40		
00502FF8	85C0	TEST EAX, EAX		
00502FFA	75 06	JNZ SHORT swdoctor.00503002		
00502FFC	C645 FF 00	MOV BYTE PTR SS:[EBP-1], 0		
00503000	EB 04	JMP SHORT swdoctor.00503006		
00503002	C645 FF 01	MOV BYTE PTR SS:[EBP-1], 1		
00503006	807D FF 00	CMP BYTE PTR SS:[EBP-1], 0		
0050300A	74 13	JE SHORT swdoctor.0050301F		
0050300C	8D55 F4	LEA EDX, DWORD PTR SS:[EBP-C]		
0050300F	B8 80305000	MOV EDI, swdoctor.00503080		
00503014	E8 77FCFFFF	CALL swdoctor.00502C90		
00503019	837D F4 00	CMP DWORD PTR SS:[EBP-C], 0		
0050301D	75 04	JNZ SHORT swdoctor.00503023		
0050301F	33C0	XOR EAX, EAX		
00503021	EB 02	JMP SHORT swdoctor.0050302F		
00503023	B0 01	MOV AL, 1		
00503025	8845 FF	MOV BYTE PTR SS:[EBP-1], AL		
00503028	33C0	XOR EAX, EAX		
0050302A	5A	POP EDI		
0050302B	59	POP ECX		
0050302C	59	POP ECX		
0050302D	64:8910	MOV DWORD PTR FS:[EAX], EDI		
00503030	68 4A305000	PUSH swdoctor.0050304A		
00503035	8D45 F4	LEA EAX, DWORD PTR SS:[EBP-C]		
00503038	B8 02000000	MOV EDI, 2		
0050303D	E8 CEE1EFFF	CALL swdoctor.00401210		
00503042	C3	RETN		

But after the call,
EAX = 0

EAX == 0이 나오는 걸 볼 수 있다.

00502FE7	85C0	TEST EAX, EAX	
00502FE9	74 11	JE SHORT swdoctor.00502FFC	
00502FEB	B8 78305000	MOV EDI, swdoctor.00503078	
00502FF0	8B45 F8	MOV EAX, DWORD PTR SS:[EBP-8]	
00502FF3	E8 48FEFFFF	CALL swdoctor.00401F40	
00502FF8	85C0	TEST EAX, EAX	
00502FFA	75 06	JNZ SHORT swdoctor.00503002	
00502FFC	C645 FF 00	MOV BYTE PTR SS:[EBP-1], 0	
00503000	EB 04	JMP SHORT swdoctor.00503006	
00503002	C645 FF 01	MOV BYTE PTR SS:[EBP-1], 1	
00503006	807D FF 00	CMP BYTE PTR SS:[EBP-1], 0	
0050300A	74 13	JE SHORT swdoctor.0050301F	
0050300C	8D55 F4	LEA EDX, DWORD PTR SS:[EBP-C]	
0050300F	B8 80305000	MOV EDI, swdoctor.00503080	
00503014	E8 77FCFFFF	CALL swdoctor.00502C90	
00503019	837D F4 00	CMP DWORD PTR SS:[EBP-C], 0	
0050301D	75 04	JNZ SHORT swdoctor.00503023	
0050301F	33C0	XOR EAX, EAX	
00503021	EB 02	JMP SHORT swdoctor.0050302F	
00503023	B0 01	MOV AL, 1	
00503025	8845 FF	MOV BYTE PTR SS:[EBP-1], AL	
00503028	33C0	XOR EAX, EAX	
0050302A	5A	POP EDI	
0050302B	59	POP ECX	
0050302C	59	POP ECX	
0050302D	64:8910	MOV DWORD PTR FS:[EAX], EDI	
00503030	68 4A305000	PUSH swdoctor.0050304A	
00503035	8D45 F4	LEA EAX, DWORD PTR SS:[EBP-C]	
00503038	B8 02000000	MOV EDI, 2	
0050303D	E8 CEE1EFFF	CALL swdoctor.00401210	
00503042	C3	RETN	

Aha, AL is set to 1 here
= good news

첫 번째 빨간 박스를 보면 두 번째 CompareText가 실행안되는 걸 볼 수 있고 밑에 코드를 좀 더 보면 MOV AL, 1을 볼 수 있다. 그럼 앞에는 코드들은 다 상관없는 걸 볼 수 있다.

00503014	E8 77FCFFFF	CALL swdoctor.00502C90	
00503019	837D F4 00	CMP DWORD PTR SS:[EBP-C], 0	
0050301D	75 04	JNZ SHORT swdoctor.00503023	
0050301F	33C0	XOR EAX, EAX	
00503021	90	NOP	
00503022	90	NOP	
00503023	B0 01	MOV AL, 1	
00503025	8845 FF	MOV BYTE PTR SS:[EBP-1], AL	
00503028	33C0	XOR EAX, EAX	

Like this

그럼 여기서 JMP분기문이 실행이 안되어야하므로 NOP으로 바꿔주면 된다.

0050301F	33C0	XOR EAX, EAX	
00503021	90	NOP	
00503022	90	NOP	
00503023	B0 01	MOV AL, 1	
00503025	8845 FF	MOV BYTE PTR SS:[EBP-1], AL	
00503028	33C0	XOR EAX, EAX	
0050302A	5A	POP EDI	
0050302B	59	POP ECX	
0050302C	59	POP ECX	
0050302D	64:8910	MOV DWORD PTR FS:[EAX], EDI	
00503030	68 4A305000	PUSH swdoctor.0050304A	
00503035	8D45 F4	LEA EAX, DWORD PTR SS:[EBP-C]	
00503038	B8 02000000	MOV EDI, 2	
0050303D	E8 CEE1EFFF	CALL swdoctor.00401210	
00503042	C3	RETN	
00503043	E9 70E1EFFF	JMP swdoctor.004011B8	
00503044	EB 02	JMP SHORT swdoctor.00503006	
00503046	8A45 FF	MOV AL, BYTE PTR SS:[EBP-1]	
00503048	85C0	TEST EAX, EAX	
0050304F	5D	POP EBP	
00503050	C2 0400	RETN 4	
00503053	0055 53	ADD BYTE PTR SS:[EBP+53], DL	

AL이 [EBP-1]에 저장되는 걸 볼 수 있고 RETN이 끝나고 다시 AL이 [EBP-1]에 저장되어 있는 값을 가져 오는 걸 볼 수 있다.

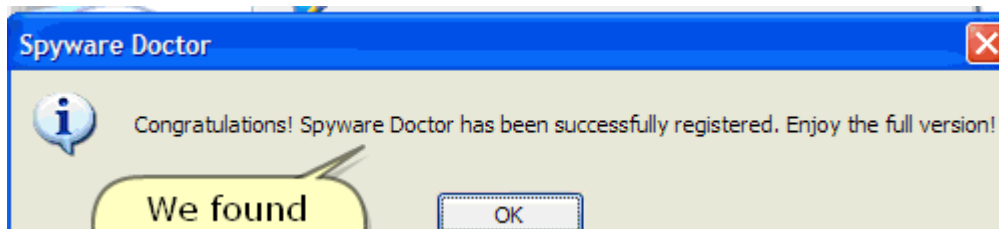
00465F71	8409	TEST AL, AL	
00465F73	75 52	JNZ SHORT swdoctor.00465FC7	
00465F75	68 4C604600	PUSH swdoctor.0046604C	
00465F7A	68 4C604600	PUSH swdoctor.0046604C	
00465F7F	E8 6CEBFFFF	CALL swdoctor.00464AF0	
00465F84	804D E4	LEA ECX, DWORD PTR SS:[EBP-10]	
00465FA7	A1 F0995000	MOV EAX, DWORD PTR DS:[F0995000]	

RETN 4까지 끝나야 내부 함수에서 나오는 걸 볼 수 있다.

00465FDB	E8 00007000	CALL swdoctor.00465FDB	
00465FDF	8B45 DC	MOV EAX, DWORD PTR SS:[EBP-24]	
00465FE2	50	PUSH EAX	
00465FE3	A1 8C985000	MOV EAX, DWORD PTR DS:[50988C]	
00465FE4	0000	MOV EAX, DWORD PTR DS:[EAX]	

Stack SS:[0012C1AC]=01A0A778, (ASCII "Congratulations! %s has been successfully registered. Enjoy the full version!")
EAX=0012C184

'F8'을 이용해서 실행해보면 0x00465FDF에서 성공 메시지가 나오는 걸 볼 수 있다.



해당 창이 나오고 'F9'를 눌러서 실행해준다.



"Buy Now"가 없어진 창이 뜨는 걸 볼 수 있다.

이제 해야 할 일은 "0x00503021" 주소 명령어를 패치(opcode 패치, "EB 02 => 90 90" == "JMP" => "NOP") 하는 로더를 만드는 것이다. 이전 레벨과 마찬가지로 "RISC

Process Pathcer"를 이용할 것이다.



위와 같이 스크립트 파일을 만들고 "RISC Process Pathcer"를 이용하여 로더를 제작하면 이제 프로그램이 실행 될 때마다 임의의 시리얼 값으로 등록이 될 것이다.

2) SpywareDocter.exe – Unpacking 사용

로더를 이용해서 언패킹을 진행하지 않고 우회할 수 있지만 **폴리모픽 코드**로 인해 언패킹을 진행하는 것이 좋다.

- 폴리모픽 코드 : 실행될 때마다 코드가 다르게 변하는 컴퓨터 코드를 의미하며, 주로 사이버 보안에서 악성코드가 백신 프로그램의 탐지를 피하기 위해 사용된다.

Address	Hex dump																ASCII
00580000	98	6E	E4	77	09	ED	E4	77	4F	83	E4	77	51	B2	E4	77	9n8w.Y8w0s8w0s8w
00580010	21	B2	E4	77	2D	6C	E4	77	B2	6F	E4	77	90	58	E4	77	888w-l8w0s8w0s8w
00580020	00	60	E4	77	35	A9	E4	77	00	00	00	00	82	00	81	7C	.8w508w...e.u
00580030	19	01	81	7C	2D	FF	80	7C	AC	92	80	7C	C7	A0	80	7C	+0u!-C!%#C!sC!
00580040	B3	9E	80	7C	39	30	82	7C	6C	02	81	7C	29	B5	80	7C	x0!90e!l0u!sC!
00580050	2F	FE	80	7C	73	B0	85	7C	9A	E1	81	7C	09	4B	83	7C	/=C!s8a!08u!Ks!
00580060	3D	0D	87	7C	6A	48	81	7C	00	1A	80	7C	81	9A	80	7C	=.C!jHu!s+C!u0C!
00580070	28	AC	80	7C	31	03	91	7C	77	1D	80	7C	40	03	91	7C	(%C!l0a!w#C!00a!
00580080	67	CC	80	7C	19	99	80	7C	67	23	80	7C	8D	2C	81	7C	g!C!+0C!g#C!l,u
00580090	EE	1E	80	7C	A9	26	82	7C	A7	24	80	7C	30	25	80	7C	-Ac!0%e!0sC!0%0C!
005800A0	3F	EB	80	7C	1B	EC	80	7C	37	97	80	7C	77	9B	80	7C	?u0C!+u0C!7u0C!w0C!
005800B0	0E	18	80	7C	8F	0C	81	7C	24	1A	80	7C	D7	EF	80	7C	8+C!A.u!s+C!i"C!
005800C0	59	2F	81	7C	E1	5B	80	7C	22	B2	80	7C	F7	0D	81	7C	u0C!s+C!00C!s+C!

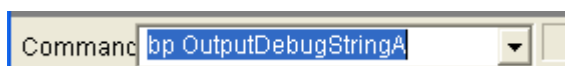
00400000	00001000	swdoctor		PE header	Imag	R	RWE
00401000	00104000	swdoctor	CODE		Imag	R	RWE
00505000	00005000	swdoctor	DATA		Imag	R	RWE
0050A000	00001000	swdoctor	BSS		Imag	R	RWE
0050B000	00015000	swdoctor	.idata		Imag	R	RWE
00520000	00010000	swdoctor	.reloc		Imag	R	RWE
00530000	00040000	swdoctor	.text	code	Imag	R	RWE
00570000	00010000	swdoctor	.adata		Imag	R	RWE
00580000	00010000	swdoctor	.data	data,import:	Imag	R	RWE
00590000	00010000	swdoctor	.reloc1	relocations	Imag	R	RWE
005A0000	00110000	swdoctor	.pdata		Imag	R	RWE
006B0000	000C1000	swdoctor	.rsrc	resources	Imag	R	RWE

D Dump - swdoctor:.idata 0050B000..0051FFFF			
0050B060	00 00 00 00	00 00 00 00	00 00 00 00
0050B070	00 00 00 00	00 00 00 00	00 00 00 00
0050B080	00 00 00 00	00 00 00 00	00 00 00 00
0050B090	00 00 00 00	00 00 00 00	00 00 00 00
0050B0A0	00 00 00 00	00 00 00 00	00 00 00 00
0050B0B0	00 00 00 00	00 00 00 00	00 00 00 00
0050B0C0	00 00 00 00	00 00 00 00	00 00 00 00
0050B0D0	00 00 00 00	00 00 00 00	00 00 00 00
0050B0E0	00 00 00 00	00 00 00 00	00 00 00 00
0050B0F0	00 00 00 00	00 00 00 00	00 00 00 00
0050B100	00 00 00 00	00 00 00 00	00 00 00 00
0050B110	00 00 00 00	00 00 00 00	00 00 00 00

프로그램을 실행하기 전에 덤프 창을 보면 IAT처럼 생긴 것처럼 보인다.

하지만 이 부분이 메인 IAT인지 스텝 IAT인지 판단을 해야한다. 대체적으로 실행했을 때 IAT가 바로 있거나, .idata섹션(물론, 메인 IAT도 여기에 있지 않을 수 있다.)에 있거나 빈 값이나 쓰레기 값으로 채워져 있으면 스텝 IAT일 경우가 높아진다.

그래서 이거는 스텝 IAT이다.



“Armadillo”패커를 사용하기 때문에 올리디버그로 실행하면 중간에 올리디버그가 꺼질 것이다. Armadillo를 사용할 때는 **“OutputDebugStringA”**를 기억해야한다. 해당 함수는 디버거를 탐지하고, 원래 코드를 암호화/복호화 하고, 바이너리 헤더를 변조하고, 원래의 IAT를 리다이렉트 시켜 변조시키기 때문이다.

Command에 "bp OutputDebugStringA"를 적어준다.(대/소문자 구분 잘해야한다.)

7C85B8E8	68 34020000	PUSH 234
7C85B8ED	68 10BC857C	PUSH kernel32.7C85BC10
7C85B8F2	E8 DF6BFAFF	CALL kernel32.7C8024D6
7C85B8F7	A1 CC56887C	MOV EAX,DWORD PTR DS:[7C8856CC]
7C85B8FC	8945 E4	MOV DWORD PTR SS:[EBP-1C],EAX
7C85B8FF	8B4D 08	MOV ECX,DWORD PTR SS:[EBP+8]
7C85B902	898D C4FDFFFF	MOV DWORD PTR SS:[EBP-23C],ECX
7C85B908	8365 FC 00	AND DWORD PTR SS:[EBP-4],0
7C85B90C	8BC1	MOV EAX,ECX
7C85B90E	8D70 01	LEA ESI,DWORD PTR DS:[EAX+1]
7C85B911	8A10	MOV DL,BYTE PTR DS:[EAX]
7C85B913	40	INC EAX
7C85B914	84D2	TEST DL,DL
7C85B916	75 F9	JNZ SHORT kernel32.7C85B911
7C85B918	2BC6	SUB EAX,ESI
7C85B91A	40	INC EAX
7C85B91B	8985 BCFDFFFF	MOV DWORD PTR SS:[EBP-244],EAX
7C85B921	898D C0FDFFFF	MOV DWORD PTR SS:[EBP-240],ECX
7C85B927	8D85 BCFDFFFF	LEA EAX,DWORD PTR SS:[EBP-244]
7C85B92D	50	PUSH EAX
7C85B92E	6A 02	PUSH 2
7C85B930	6A 00	PUSH 0
7C85B932	68 06000140	PUSH 40010006
7C85B937	E8 4576E8EE	CALL kernel32.RaiseException
7C85B93C	E9 52020000	JMP kernel32.7C85BB93
7C85B941	90	NOP
7C85B942	90	NOP
7C85B943	90	NOP
7C85B944	90	NOP
7C85B945	90	NOP
7C85B946	33C0	XOR EAX,EAX
7C85B948	40	INC EAX
7C85B949	C3	RETN
7C85B94A	90	NOP
7C85BB93	834D FC FF	OR DWORD PTR SS:[EBP-4],FFFFFFFF
7C85BB97	8B4D E4	MOV ECX,DWORD PTR SS:[EBP-1C]
7C85BB9A	E8 0BDCFAFF	CALL kernel32.7C8097AA
7C85BB9E	E8 6D69EAFE	CALL kernel32.7C802511
7C85BBA4	C2 0400	RETN 4

0x7C85B8E8에서 멈추는 걸 볼 수 있다. 해당 함수가 끝나긴 위해서는 밑에 RETN이 실행되어야한다. 하지만 그 중간에 있는 JMP 분기문으로 인해서 0x7C85BB93으로 넘어가서 RETN4를 반환하는 걸 볼 수 있다. 중간에 있는 코드로 인해서 OutputDebugStringA가 실행되므로 아예 실행안되게 처음 부분인 PUSH 234 -> RETN 4로 변경해준다.

7C85B8E8	C2 0400	RETN 4
7C85B8EB	90	NOP
7C85B8EC	90	NOP
7C85B8ED	68 10BC857C	PUSH kernel32.7C85BC10
7C85B8F2	E8 DF6BFAFF	CALL kernel32.7C8024D6

이렇게 변경해주면 "Armadillo" 프로텍터의 디버거 탐지는 무력화 되었다. 이제 OEP를 찾을 것인데 "CreateThread" 주소에 BP를 설치하여 OEP를 찾을 수 있다.

Command

B Breakpoints			
Address	Module	Active	Disassembly
7C810707	kernel32	Always	MOV EDI,EDI
7C85B8E8	kernel32	Always	RETN 4

Command에 "bp CreateThread"를 쓰고 BP를 보면 0x7C810707이 걸린 걸 볼 수 있다. 해당 부분으로 넘어가서 코드를 살펴본다.

※ CreateThread에 BP를 거는 이유

대부분의 패커는 언패킹(복호화) 루틴이 끝나면 새 스레드를 만들어 그 스레드의 시작 함수로 실제 원본 코드(OEP)를 실행시키는 식으로 제어를 옮기기 때문이다. 즉, CreateThread는 **"언패킹이 끝나고 진짜 코드로 넘어가는 순간"**을 잡을 수 있는 아주 유력한 지점인 거다. -> 하지만 모든 패커가 CreateThread함수를 쓰는 것이 아니다. 다른 함수를 사용하거나 아예 다른 방법으로 제어를 옮길 수 있다.

7C810707	8BFF	MOV EDI,EDI
7C810709	55	PUSH EBP
7C81070A	8BEC	MOV EBP,ESP
7C81070C	FF75 1C	PUSH DWORD PTR SS:[EBP+1C]
7C81070F	FF75 18	PUSH DWORD PTR SS:[EBP+18]
7C810712	FF75 14	PUSH DWORD PTR SS:[EBP+14]
7C810715	FF75 10	PUSH DWORD PTR SS:[EBP+10]
7C810718	FF75 0C	PUSH DWORD PTR SS:[EBP+8]
7C81071B	FF75 08	PUSH DWORD PTR SS:[EBP+4]
7C81071E	6A FF	PUSH -1
7C810720	E8 D7FDFFFF	CALL kernel32.CreateRemoteThread
7C810725	5D	POP EBP
7C810726	C2 1800	RETN 18

BP를 0x7C810726에 걸어주고 0x7C810707의 BP는 제거해준다.

왜냐하면 Armadillo는 API 진입 지점(BP가 흔히 걸리는 위치)을 검사하기 때문이다. 그래서 API 진입점이 아닌 "API 호출이 끝난 직후 지점"은 Armadillo가 검사하지 않기 때문에 RETN 18에 BP를 걸어준다. RETN 18에서 'F8'을 눌러 CreateThread함수에서 나온다.

※ CreateThread & CreateRemoteThread의 차이

CreateThread : 같은 프로세스 안에서 새로운 스레드를 만든다.

CreateRemoteThread : 다른 프로세스에 새 스레드를 만들어 그쪽 메모리에서 실행시킨다.

- 패커가 일반적으로 하는 절차

1. 파일을 실행하면 **스텝(stub)**이 먼저 실행된다.

스텝 = 언패킹을 담당하는 작은 프로그램(언패커)

2. 스텝이 메모리에 코드를 복원하려면 새 메모리 공간이 필요하다.

이때 VirtualAlloc 또는 NtAllocateVirtualMemory 같은 함수를 호출해서 공간을 만든다.

비유: 창고(메모리)를 빌리는 것

3. 복원된 코드를 그 공간에 쓴다. (WriteProcessMemory 유사 동작 또는 단순 메모리 쓰기)

비유: 창고 바닥에 물건(코드)을 꺼내어 정리하는 것

4. 그 공간에 실행 권한(EXECUTE)을 준다. (VirtualProtect 또는 NtProtectVirtualMemory)

비유: 창고 문에 출입허가(운영 허가) 스티커 붙이는 것!!!

5. 마지막으로 그 코드(창고)를 실행시킨다.

보통 자기 프로세스면 CreateThread(lpStartAddress)

다른 프로세스면 CreateRemoteThread(hProcess, remoteAddr) 같은 걸로 실행

이 순간이 바로 OEP(실제 원본 코드가 '정식으로' 실행되는 시점)이다.

-> 굳이 새 메모리를 쓰는 이유

원본 코드(암호화된/압축된 상태)를 깨끗하게 복원해서 실행해야 하기 때문이다.

파일(.exe) 내부를 바로 바꾸면 탐지/무결성 문제 생길 수 있고, 실행 중에 빨리 바뀌어야 하니까 메모리를 이용하는 게 편하다.

또 메모리에 복원하면 실행권한만 주면 바로 실행 가능하니까 디스크 접근보다 빠르고 안전하게 동작한다.

00E7114A	5E	POP ESI	00E8C3E5
00E7114B	C9	LEAVE	
00E7114C	C3	RETN	
00E7114D	55	PUSH EBP	
00E7114E	8BEC	MOV EBP,ESP	
00E71150	81EC 28010000	SUB ESP,128	
00E71156	56	PUSH ESI	swdoctor.00586A30
00E71157	57	PUSH EDI	
00E71158	BE 0075E900	MOV ESI,0E97500	ASCII "MainClass"
00E7115D	8DBD D8FEFFFF	LEA EDI,DWORD PTR SS:[EBP-128]	
00E71160	0F	MOV EBP,DWORD PTR ECX+50013	

RETN 18이 0x00E7114A로 넘어가고 'F8'을 눌러서 RETN까지 넘어온다.

00E8C3E5	A1 F81EEA00	MOV EAX,DWORD PTR DS:[EA1EF8]	
00E8C3EA	59	POP ECX	
00E8C3EB	8B48 68	MOV ECX,DWORD PTR DS:[EAX+68]	kernel32.7C8107FD
00E8C3EE	3348 64	XOR ECX,DWORD PTR DS:[EAX+64]	
00E8C3F1	3348 64	XOR ECX,DWORD PTR DS:[EAX+64]	
00E8C3F4	F6	TEST ECX,ECX	
00E8C3F7	75	JNZ 00E8C401	
00E8C3F9	6A 01	PUSH 1	
00E8C3FB	E8 F7B8FDFF	CALL 00E67CF7	
00E8C400	59	POP ECX	kernel32.7C8107FD
00E8C401	53	PUSH EBX	
00E8C402	C705 E079E900 C087E900	MOV DWORD PTR DS:[E979E0],0E987C0	ASCII "RC"
00E8C40C	E8 4CE5FDFF	CALL 00E6A95D	
00E8C411	59	POP ECX	kernel32.7C8107FD
00E8C412	F8 B5F2FFFF	CALL 00E7D6C0	

다음 'F8'을 눌러서 넘어가면 해당 주소로 넘어가고 좀 더 트레이싱하면 OEP를 찾을 수 있다.

00E8C47B	3348 20	XOR ECX,DWORD PTR DS:[EAX+20]	
00E8C47E	2BF9	SUB EDI,ECX	
00E8C480	FFD7	CALL EDI	swdoctor.00503F68
00E8C482	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	swdoctor.00580370
00E8C485	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00E8C488	5F	POP EDI	swdoctor.00400000

'F8'을 이용해서 계속 실행하다보면 해당 부분에서 0x00503F68로 넘어가는 걸 볼 수 있다. 해당 부분이 OEP로 넘어가는 부분인 걸 볼 수 있다.

00503F68	55	PUSH EBP	
00503F69	8BEC	MOV EBP,ESP	
00503F6B	83C4 F	ADD ESP,-1C	
00503F6E		PUSH EBX	
00503F6F		PUSH ESI	
00503F71		PUSH EDI	
00503F72		XOR EAX,EAX	
00503F73		MOV DWORD PTR SS:[EBP-1C],EAX	swdoctor.00586A30
00503F74		MOV DWORD PTR SS:[EBP-18],EAX	swdoctor.00503F68
00503F75		MOV DWORD PTR SS:[EBP-14],EAX	swdoctor.00580370
00503F76		MOV EAX,swdoctor.005037E8	swdoctor.00580370
00503F77		CALL swdoctor.0040147C	swdoctor.00580370
00503F78		XOR EAX,EAX	
00503F79		PUSH EBP	
00503F7A			
00503F7B			
00503F7C			
00503F7D			
00503F7E			
00503F7F			
00503F80			
00503F81			
00503F82			
00503F83			
00503F84			
00503F85			
00503F86			
00503F87			
00503F88			

'F7'을 이용해서 그 함수로 들어가보면 OEP를 볼 수 있다. 해당 부분에 HW BP를 걸어준다.

대부분의 사람들은 OEP를 찾았으니 이제 해당 바이너리를 덤프하려고 할 것이다. 그러나 덤프를 하지 않을 것이다. 왜냐하면 위 사진처럼 메모리 맵 창에서 PE 헤더를 보면 헤더가 없다고 나온다. 이런 현상이 발생하는 이유는 "Armadillo" 프로텍터가 PE 헤더 오프셋을 삭제해서 헤더를 손상시켰기 때문이다. 이런 행위는 바이너리를 덤프하거나 IAT를 추가하는 것을 막기 위해 사용된다.

이런 행위를 우회하는 간단한 방법이 있다. 또 다른 디버거를 실행하고 원본 exe에 존재하는 헤더를 "Binary Copy" 한 뒤 붙여넣으면 된다. 우선 "Import RECT"툴을 이용하여 "IAT"를 찾을 수 있는지 부터 알아 볼 것이다.

IAT Infos Needed

OEP

RVA

Size

AutoSearch

Get Imports

IAT Infos Needed

OEP

RVA

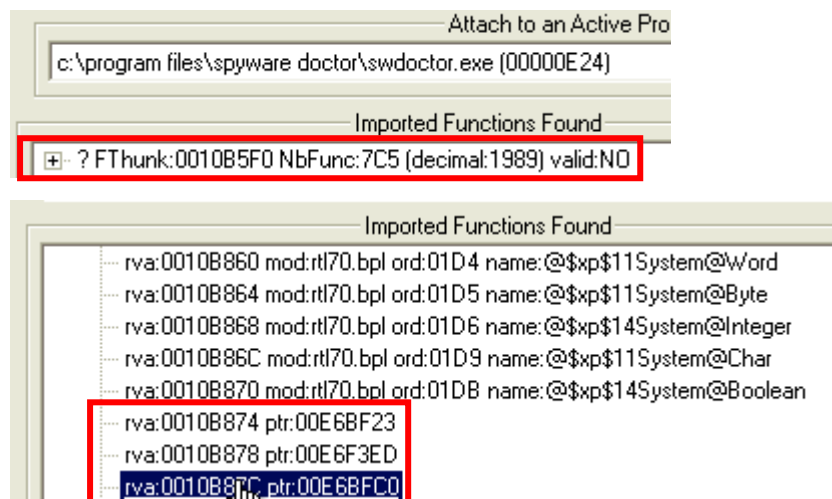
Size

AutoSearch

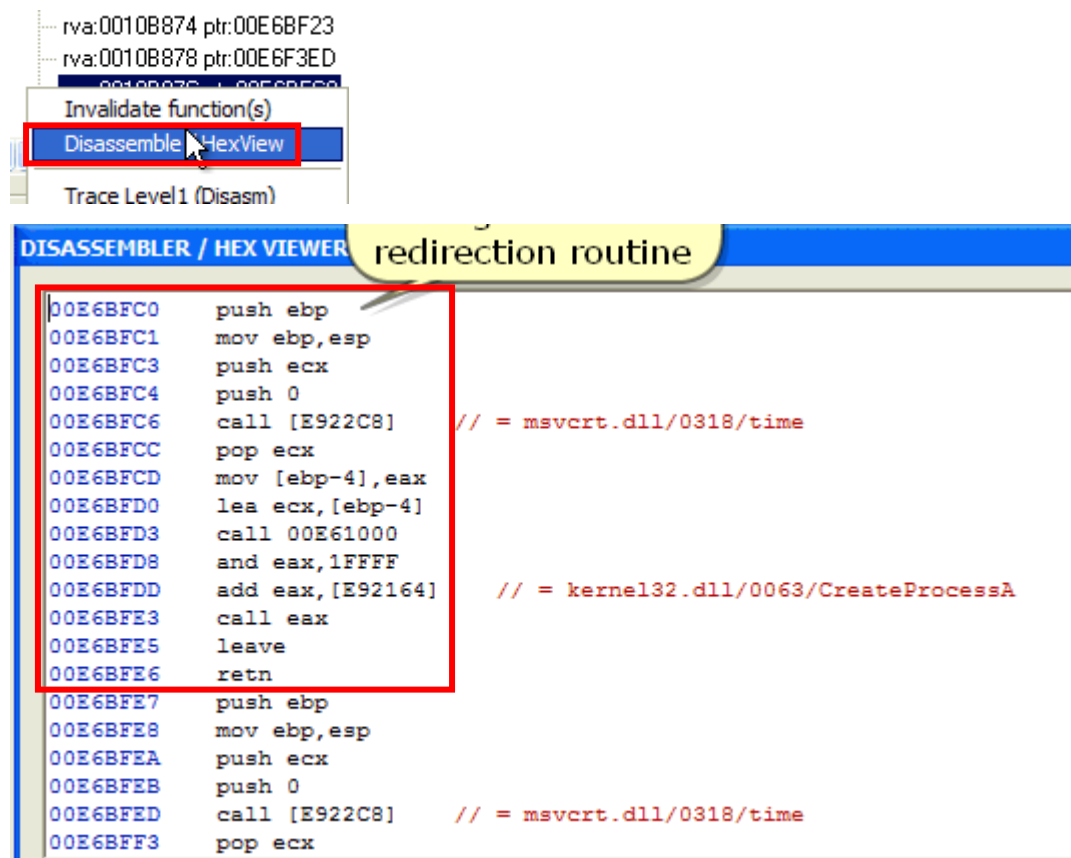
Get Imports

"ImportREC" 툴에서 OEP를 변경하고 "AutoSearch"를 이용하여 나머지 값을 자동으로 세

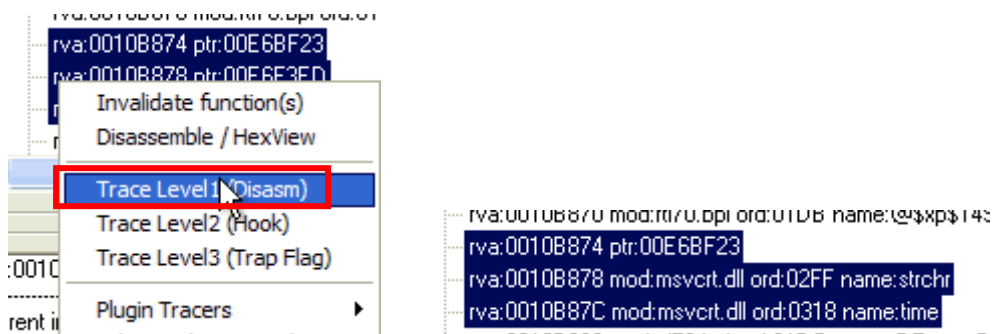
팅 하면 우측의 사진처럼 된다.



이렇게 3개 때문에 valid:NO가 나오는 걸 볼 수 있다.



API를 호출하는데 위 사진과 같은 부분이 나타나고 별 명령어 없이 리턴 된다. 추측해보면 "API Redirect"를 이용하여 특정 API 함수 주소를 다른 곳으로 돌려 버린 것이다.



좀 더 분석하기 위해, "Trace Level1" 기능을 이용하였지만 큰 도움이 되지 않기 때문에 "Magic Jump"를 찾을 것이다.

- Magic jump: 패커가 API 호출을 가로채서 다른 코드(자기 스텝/리다이렉션 루틴)로 점프시키는 트릭을 말함!!!

+ "Armadillo" 프로텍터는 실제 API가 아니라 그냥 다른 주소로 분기시키는 API를 인젝션 하기도 한다. 해당 API 부터 찾아본다.

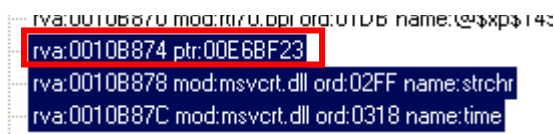
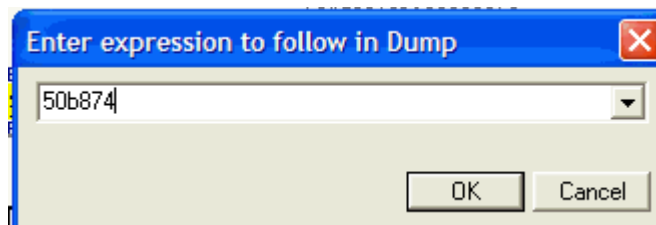


Image Base + rva == IAT 위치



덤프 창에서 해당 부분을 들어가준다.

Address	Hex dump
0050B874	23 BF E6 00 ED F3 E6 00
0050B878	04 9E 00 40 54 9E 00 40
0050B87C	40 9D 00 40 00 9D 00 40
0050B880	8E BD D1 77 0C 94 D1 77

해당 부분인 0x00E6BF23을 코드에서 살펴본다.



위 루틴은 그냥 아무것도 하지 않는 루틴이다. 처음에 EBX 레지스터를 스택에 "push" 하

지만 리턴 되기 전에 "pop" 한다. "push / pop" 코드 사이에는 그냥 루프만이 존재한다. 이런 API 가 바로 프로텍터에서 인젝션 한 쓸데 없는 API 이다.

이제 바이너리를 재실행하여 "Magic Jump"를 찾을 것이다.

Command	bp OutputDebugStringA		
7C85B8E8	C2 0400	RETN 4	
7C85B8EB	90	NOP	
7C85B8EC	90	NOP	
7C85B8ED	68 10BC857C	PUSH kernel32.7C85BC10	
7C85B8F2	E8 DF6BFAFF	CALL kernel32.7C8024D6	

재실행하기 때문에 다시 바꿔주고 실행해야한다.

Command	bp GetModuleHandleA
---------	---------------------

Command에 "bp GetModuleHandleA"를 입력하고 BP를 걸어준다.

※ 패커가 이 함수를 이용해 모듈 베이스를 얻어 IAT를 만들거나 리다이렉션(트램폴린, magic jump)을 설정하기 때문이다.

B Breakpoints			
Address	Module	Condition	Disassembly
7C80B529	kernel32	Always	MOV EDI,EDI

BP 리스트 들어가서 해당 부분으로 넘어가준다.

7C80B529	8BFF	MOV EDI,EDI	
7C80B52B	55	PUSH EBP	
7C80B52C	8BEC	MOV EBP,ESP	
7C80B52E	837D 08 00	CMPL DWORD PTR SS:[EBP+8],0	
7C80B532	74 18	JE SHORT kernel32.7C80B54C	
7C80B534	FF75 08	PUSH DWORD PTR SS:[EBP+8]	
7C80B537	E8 682D0000	CALL kernel32.7C80E2A4	
7C80B53C	85C0	TEST EAX,EAX	
7C80B53E	74 08	JE SHORT kernel32.7C80B548	
7C80B540	FF70 04	PUSH DWORD PTR DS:[EAX+4]	
7C80B543	E8 F4300000	CALL kernel32.GetModuleHandleW	kernel32.7C800000
7C80B548	5D	POP EBP	00E8367D
7C80B549	C2 0400	RETN 4	

0x7C80B529 BP를 제거해주고 RETN 4에 BP를 걸고 넘어가준다. 'Alt+F9'을 눌러서 해당 함수에 나온다.

00E83677	FF15 D820E900	CALL DWORD PTR DS:[E920D81]	kernel32.GetModuleHandleA
00E83670	50	PUSH EAX	kernel32.7C800000
00E8367E	E8 7F7DFEFF	CALL 00E6B402	00E98744
00E83683	59	POP ECX	00E98744
00E83684	59	POP ECX	kernel32.7C800000
00E83685	A3 6C20EA00	MOV DWORD PTR DS:[EA206C],EAX	kernel32.7C800000
00E8368A	68 3887E900	PUSH 0E98738	ASCII "VirtualFree"
00E8368F	50 7F7DFEFF	CALL 00E6B402	kernel32.7C800000
00E83694	FF15 D820E900	CALL DWORD PTR DS:[E920D81]	kernel32.GetModuleHandleA
00E8369A	50	PUSH EAX	kernel32.7C800000
00E8369B	E8 627DFEFF	CALL 00E6B402	00E98744
00E836A0	59	POP ECX	00E98744
00E836A1	59	POP ECX	kernel32.7C800000
00E836A2	A3 7020EA00	MOV DWORD PTR DS:[EA2070],EAX	kernel32.7C800000
00E836A7	50	PUSH EAX	kernel32.7C800000
00E836A8	F7D0	NOT EAX	kernel32.7C800000
00E836AA	0FC8	BSWAP EAX	kernel32.7C800000
00E836AC	58	POP EAX	00E98744
00E836AD	72 00	JNZ SHORT 00E836AF	

GetModuleHandleA에서 나오면 0x00E83670으로 넘어가고 두 번째 GetModuleHandleA에서 다시 넘어가는 걸 볼 수 있다. 그럼 똑같이 BP 걸리는 곳에 멈추는 걸 알 수 있다.

00E6AC0C	50	PUSH EAX	kernel32.7C800000
00E6AC0D	FF15 D820E900	CALL DWORD PTR DS:[E920D81]	kernel32.GetModuleHandleA
00E6AC13	8B0D E4C9E900	MOV ECX, DWORD PTR DS:[E9C9E41]	kernel32.7C800000
00E6AC19	89040E	MOV DWORD PTR DS:[ESI+ECX],EAX	kernel32.7C800000
00E6AC1C	A1 E4C9E900	MOV EAX, DWORD PTR DS:[E9C9E41]	
00E6AC21	391C06	CMP DWORD PTR DS:[ESI+EAX],EBX	
00E6AC24	75 16	JNZ SHORT 00E6AC3C	
00E6AC26	8D85 B4FEFFFF	LEA EAX, DWORD PTR SS:[EBP-14C]	
00E6AC2C	50	PUSH EAX	kernel32.7C800000
00E6AC2D	FF15 E020E900	CALL DWORD PTR DS:[E920E01]	kernel32.LoadLibraryA
00E6AC33	8B0D E4C9E900	MOV ECX, DWORD PTR DS:[E9C9E41]	kernel32.7C800000
00E6AC39	89040E	MOV DWORD PTR DS:[ESI+ECX],EAX	
00E6AC3C	A1 E4C9E900	MOV EAX, DWORD PTR DS:[E9C9E41]	
00E6AC41	391C06	CMP DWORD PTR DS:[ESI+EAX],EBX	
00E6AC44	0F84 35010000	JE 00E6AD7F	

그럼 두 번째 GetModuleHandleA에서 나오고 0x00E6AC44인 JE분기문이 Magic Jump인 걸 알 수 있다. 왜냐하면 LoadLibraryA 때문이다.

JE -> JMP로 바꿔준다. 왜냐하면 모든 API 들은 리다이렉트 되지 않고 원본 주소를 가르키기 때문이다.

- LoadLibraryA : 지정한 DLL을 로드해서 모듈 핸들(베이스 주소)을 반환(존재하면 그걸 반환). 런타임에 모듈을 불러오고 IAT 결합을 유도할 때 사용된다.

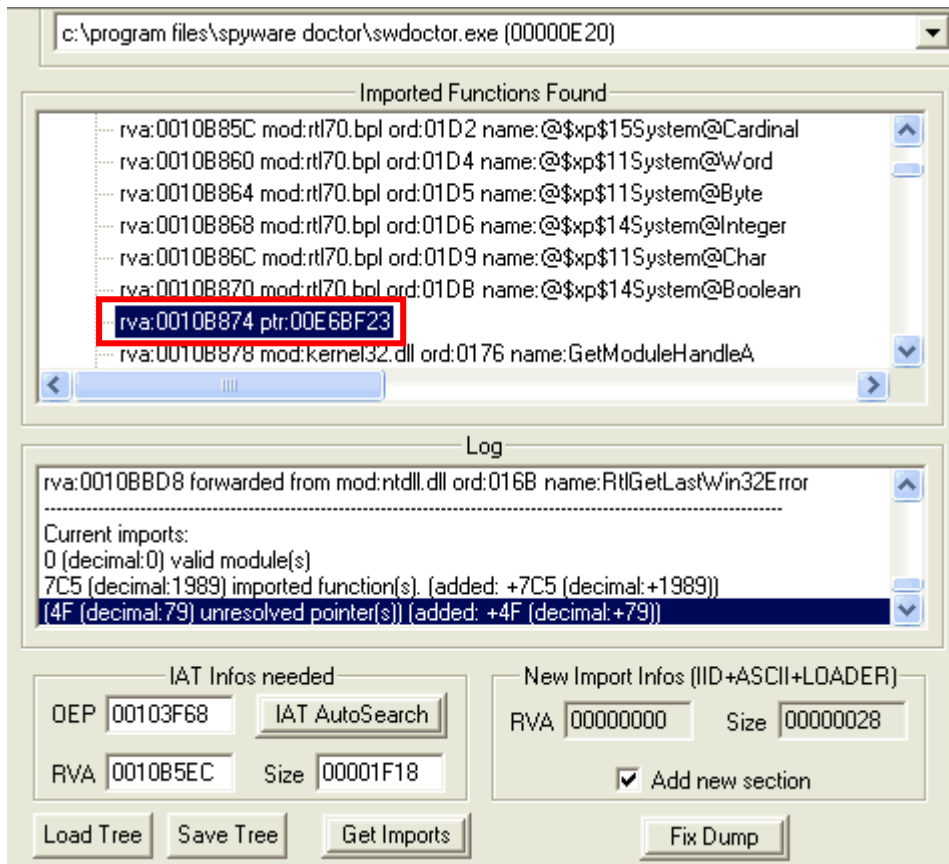
즉, GetModuleHandleA로 프로세스에 이미 로드된 모듈의 베이스 주소(모듈 핸들)를 반환하고 LoadLibraryA로 IAT를 만들기 때문이다.

00503F68	55	PUSH EBP	
00503F69	8BEC	MOV EBP, ESP	
00503F6B	83C4 E4	ADD ESP, -1C	
00503F6E	53	PUSH EBX	
00503F6F	56	PUSH ESI	
00503F70	57	PUSH EDI	

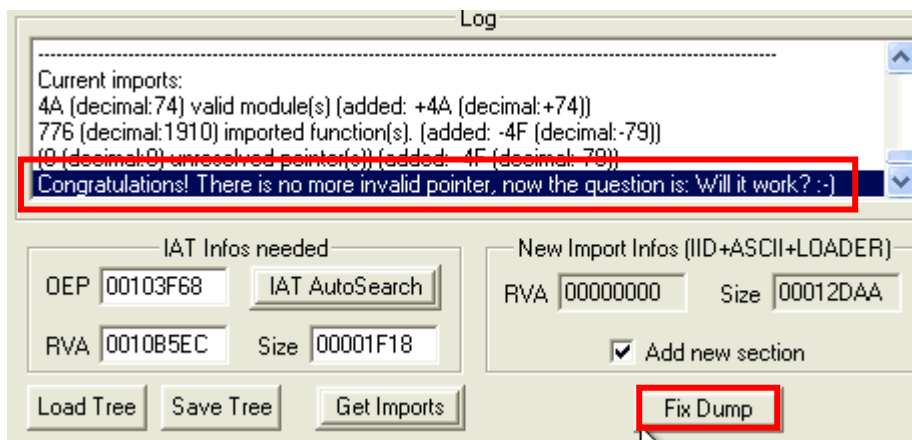
분기문을 패치하고 덤프를 쓰기 위해 OEP로 이동한다. 덤프는 "Lord PE"툴을 이용할 것이다.

Path	PID	ImageBase	ImageSize
c:\reverseit\programs\importrec\importrec.exe	000009C4	00400000	0007C200
c:\program files\instantdemo\instantdemo.exe	00000FD0	00400000	00185000
swdoctor.exe	00000E20	00400000	00371000
be.exe	000002F8	00400000	00036000

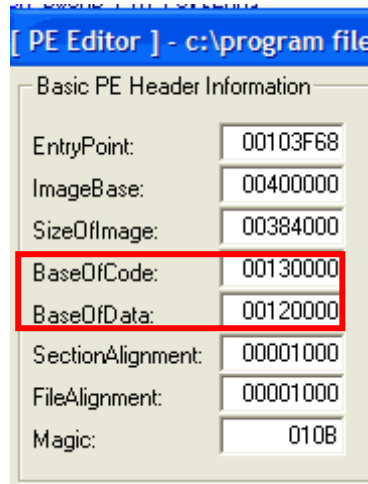
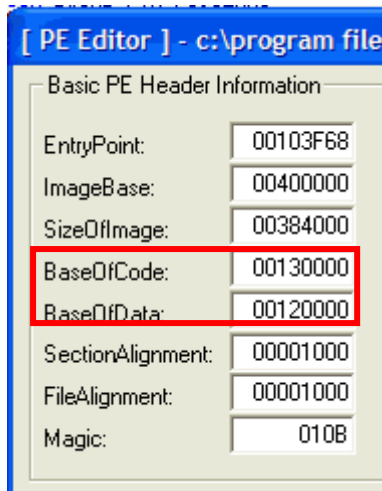
덤프를 뗐으니 이제 다시 “Impot REC” 틀에서 IAT를 고쳐주어야 한다.



“Import REC”틀에서 대상 바이너리를 열고 “OEP” 값을 이전에 찾았던 값으로 변경한 “Auto Search – Get Import” 하면 이전과 동일하게 한 개의 모듈 상태가 “valid : NO” 일 것이다. 그러나 모듈 내에서 문제되는 부분들은 쓸모 없는 API들이다. (패치를 통하여 API 리다이렉션이 실행 되지 않기 때문에 지금 문제되는 부분은 프로텍터가 인젝션 한 API일 것이다.)

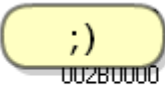


LordPE로 덤프 뜯은 파일에 덮어준다.



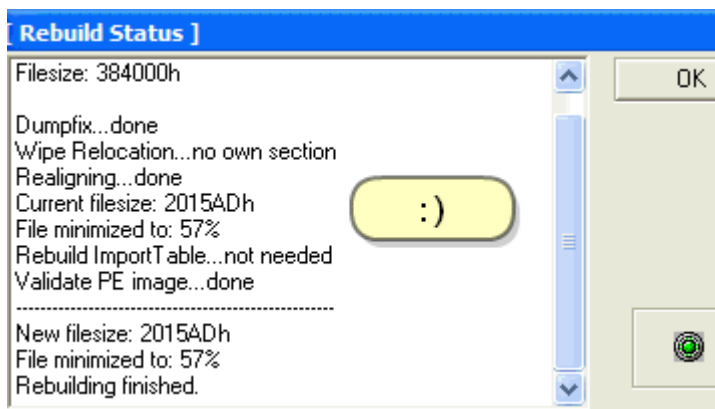
Name	VOffset	VSize	ROffset	RSize	Flags
CODE	00001000	00103304	00001000	00103304	60000020
DATA	00105000	00004A7C	00105000	00004A7C	C0000040

LordPE를 이용해서 PE를 수정해준다.

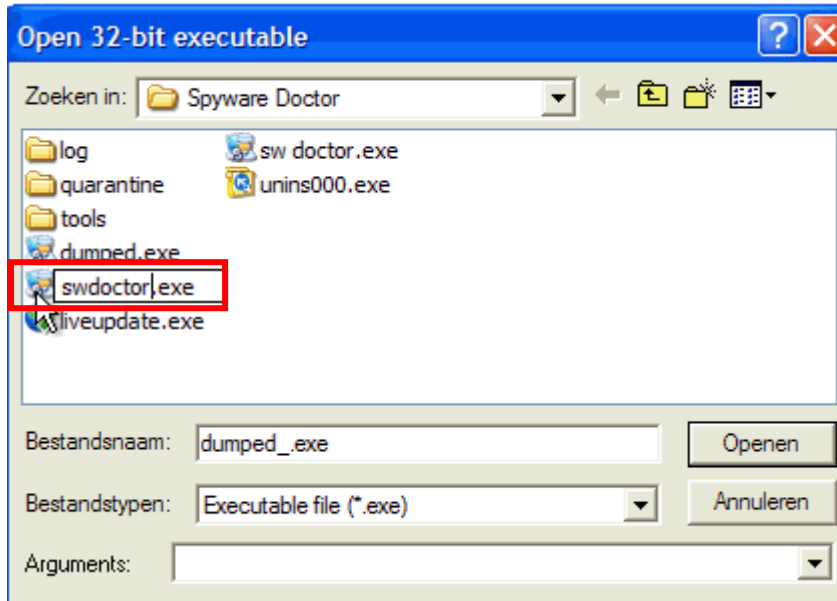


Name	VOffset	VSize	ROffset	RSize	Flags
CODE	00001000	00103304	00001000	00103304	60000020
DATA	00105000	00004A7C	00105000	00004A7C	C0000040
BSS	0010A000	00000A79	0010A000	00000A79	C0000000
.idata		00014D38	0010B000	00014D38	C0000040
.reloc		0000FB28	00120000	0000FB28	50000040
.rsrc	002B0000	000C1000	002B0000	000C1000	50000040
.mact	00371000	00013000	00371000	00013000	E0000060

그리고 용량을 줄이기 위해 필요없는 섹션도 삭제해주고 Rebuild PE를 눌러준다.



용량이 줄어든 걸 확인할 수 있다.



Armadillo가 아니라 프로그램 자체가 분석자를 속이기 위한 마지막 트릭이다.
'이름(name)'을 검사하는 루틴이 있어서, 파일 이름이 바뀌면 오류가 발생한다. (원본 파일에서도 마찬가지다.)

따라서 덤프한 파일의 이름을 원래 파일 이름으로 바꿔줘야한다.

그럼 실행되는 걸 볼 수 있다.